

autograd

CSCI
270

torch's superpower is its
ability to build a function
graph and calculate its
partial derivatives

the function graph for linear regression

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])

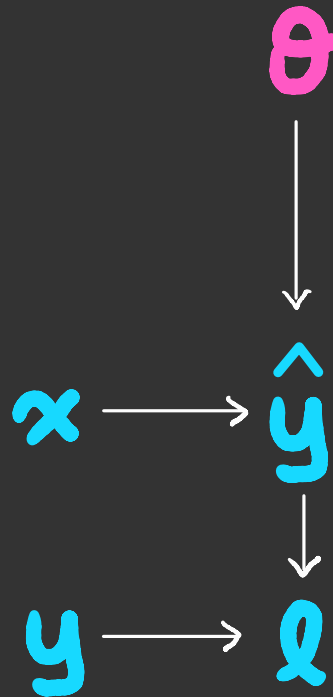
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()

loss = ((y_hat - y) ** 2).sum()

loss.backward()

print(f"d loss/d theta = {theta.grad}")
```



we provide values for the root nodes

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])
```

```
y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
```

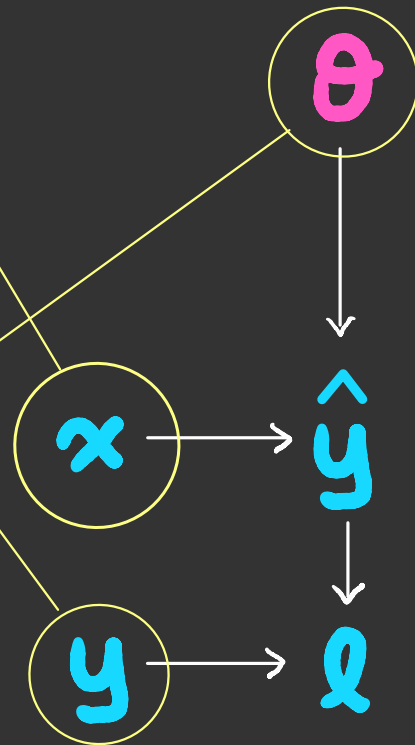
```
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)
```

```
y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
```

```
loss = ((y_hat - y) ** 2).sum()
```

```
loss.backward()
```

```
print(f"d loss/d theta = {theta.grad}")
```



we inform torch that we will want derivatives of the form $\frac{\partial}{\partial \theta}$

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])

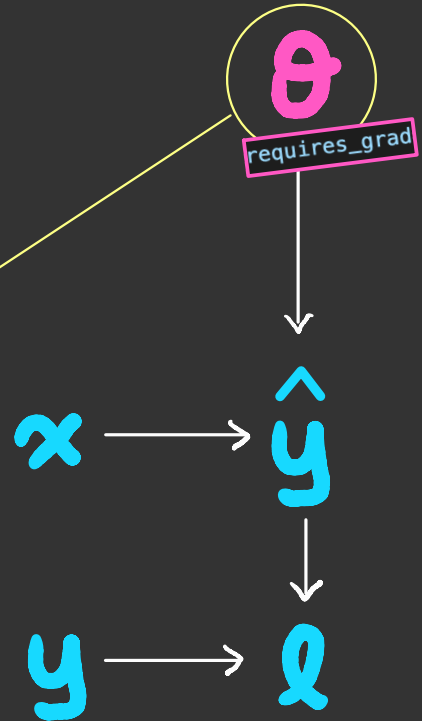
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()

loss = ((y_hat - y) ** 2).sum()

loss.backward()

print(f"loss/d theta = {theta.grad}")
```



the predictions ($\hat{y}$) are a function of θ and x

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])

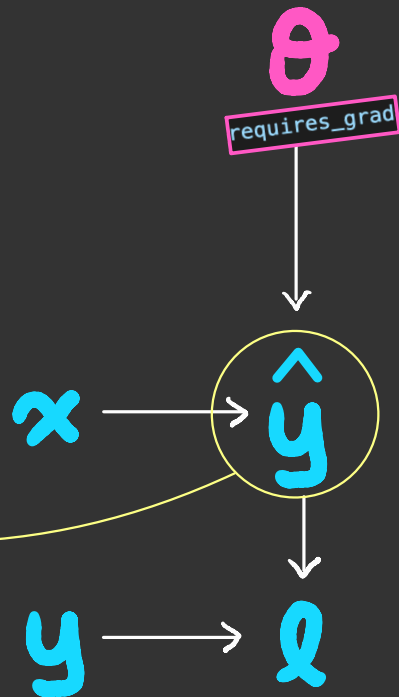
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()

loss = ((y_hat - y) ** 2).sum()

loss.backward()

print(f"d loss/d theta = {theta.grad}")
```



the loss is a function of $\hat{y}$ and y

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])

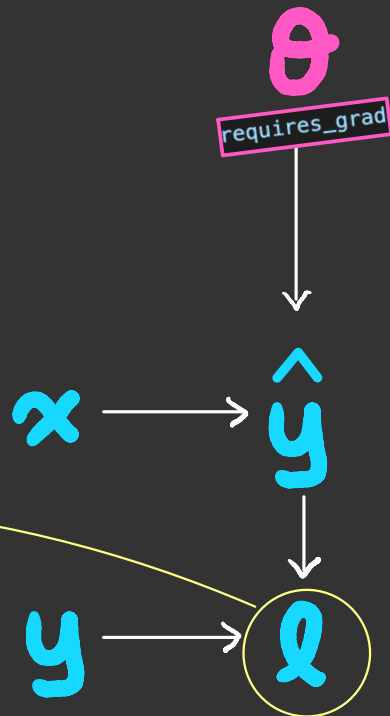
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()

loss = ((y_hat - y) ** 2).sum()

loss.backward()

print(f"d loss/d theta = {theta.grad}")
```



we inform torch to compute all derivatives of the form $\frac{\partial \ell}{\partial \mathbf{a}}$ (where $\mathbf{a}$ has `requires_grad=True`)

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

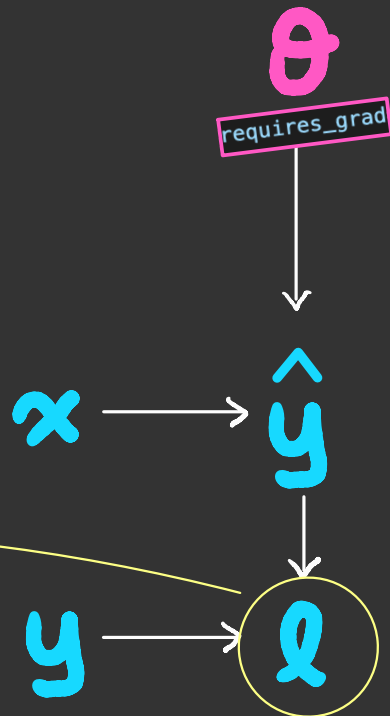
y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])

theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()

loss = ((y_hat - y) ** 2).sum()

loss.backward()
print(f"d loss/d theta = {theta.grad}")
```



loss increases as
we increase θ_1

loss increases as
we increase θ_2

loss increases as
we increase θ_3

d loss/d theta = tensor([90.0000, 559.6000, 29320.0000])

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

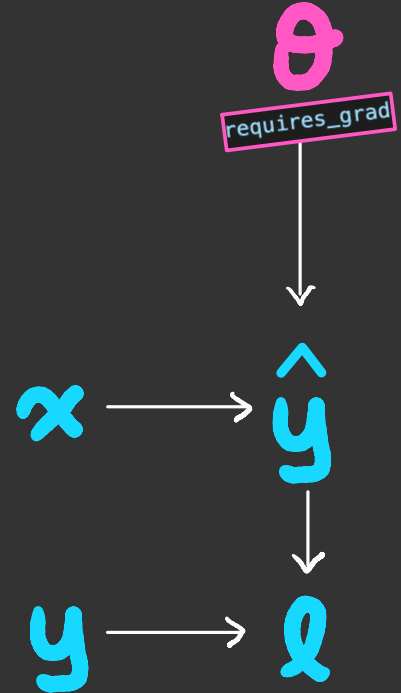
y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])

theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()

loss = ((y_hat - y) ** 2).sum()

loss.backward()
print(f"d loss/d theta = {theta.grad}")
```



we might try to implement gradient descent this way

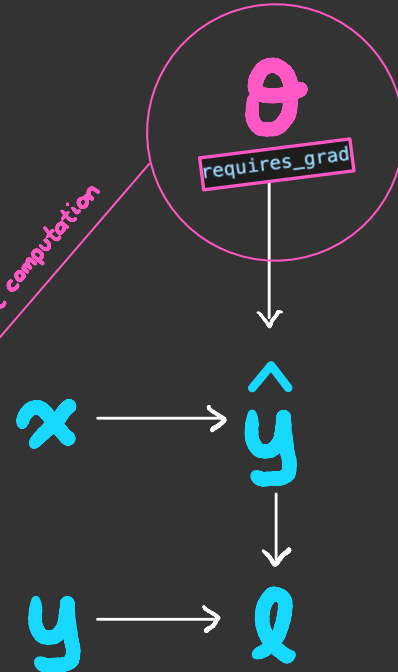
```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    theta = theta - learning_rate * theta.grad
```

update θ after each gradient computation



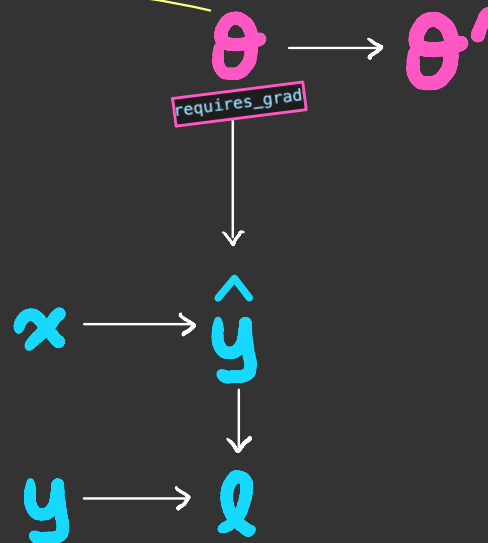
but this assignment actually creates
a new node in the function graph

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    theta = theta - learning_rate * theta.grad
```

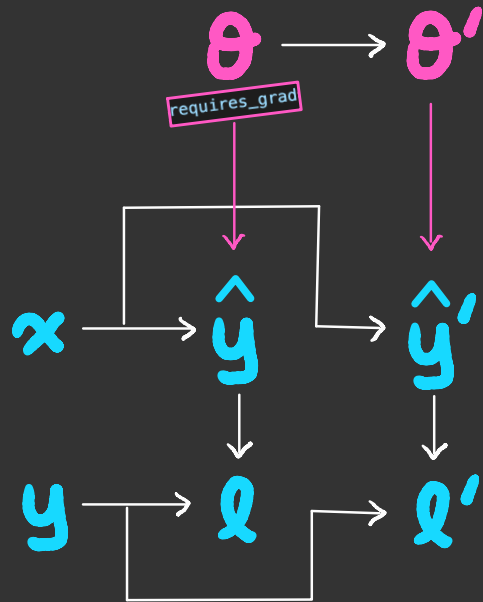


so on the next iteration, new versions of $\hat{y}$ and l are created

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])
y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

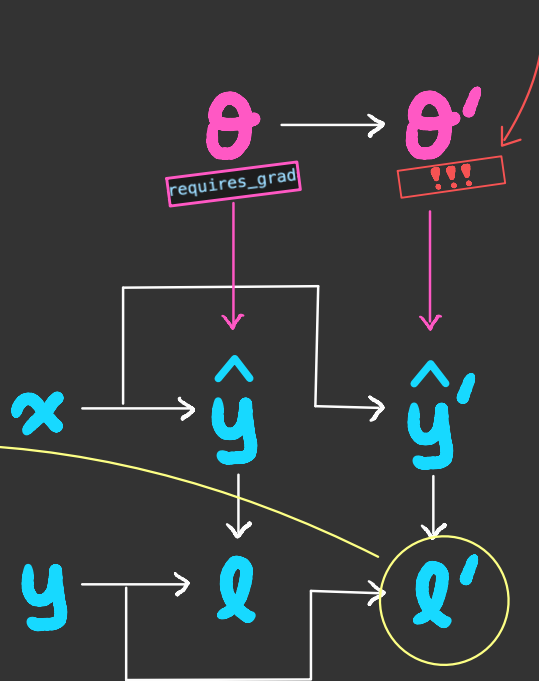
    theta = theta - learning_rate * theta.grad
```



but the new θ doesn't have `requires_grad=True`

```
x = torch.tensor([
    [
        [1, 1, 1, 1, 1, 1],
        [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
        [120, 200, 120, 250, 260, 150],
    ]
])
y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()
    theta = theta - learning_rate * theta.grad
```



so this doesn't work

we can patch this

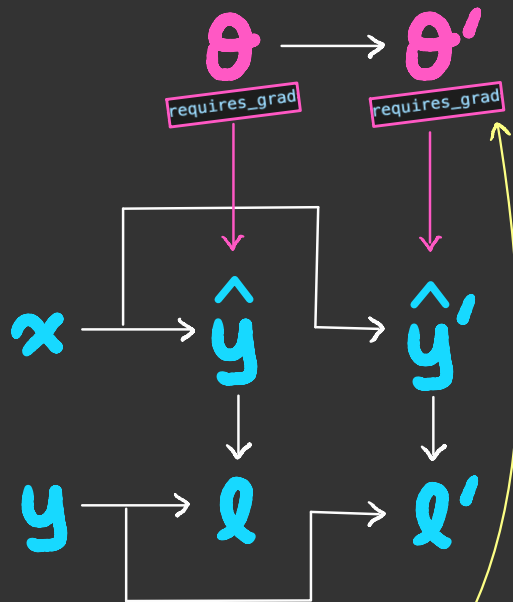
```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    theta = theta - learning_rate * theta.grad

    theta.retain_grad()
```



but the function graph keeps growing

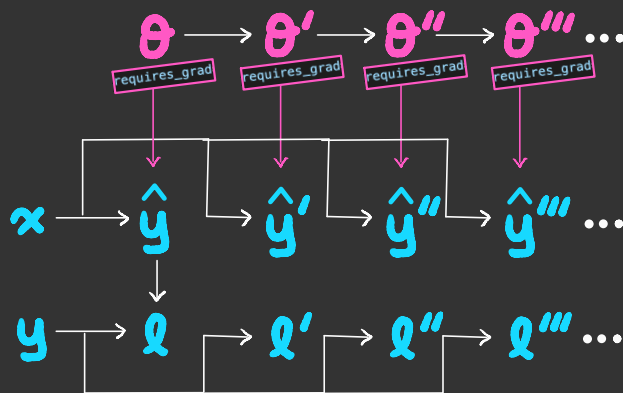
```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    theta = theta - learning_rate * theta.grad

    theta.retain_grad()
```



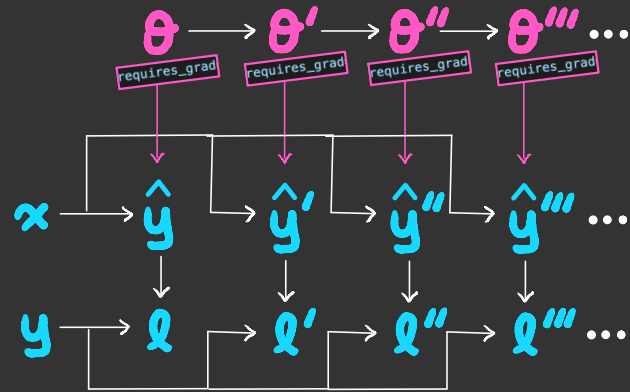
we can adjust the code so that

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    theta = theta - learning_rate * theta.grad
    theta.retain_grad()
```



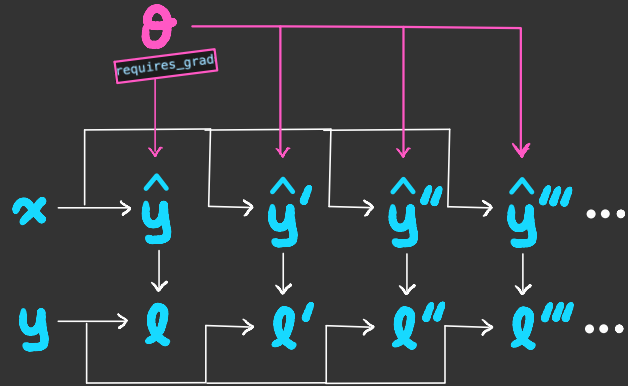
it doesn't create a new node

we can adjust the code so that

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])
y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    theta -= learning_rate * theta.grad
```



it doesn't create a new node

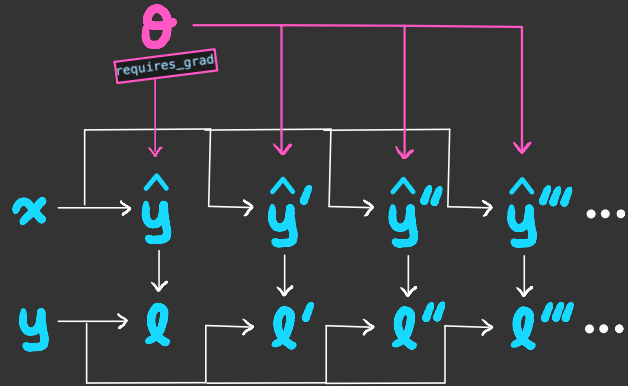
but torch doesn't allow us to

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    theta -= learning_rate * theta.grad
```



update the value of existing nodes

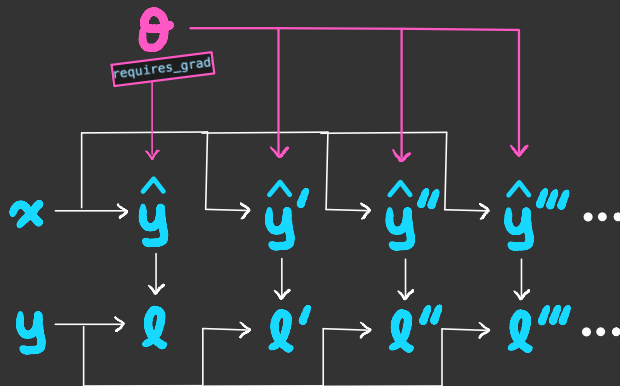
unless we provide explicit consent

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])

y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    with torch.no_grad():
        theta -= learning_rate * theta.grad
```

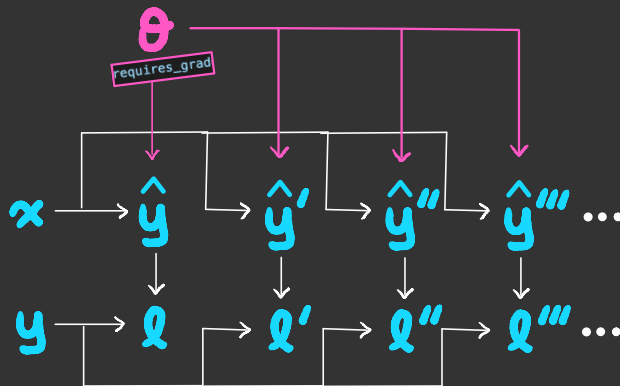


unless we provide explicit consent

```
x = torch.tensor([
    [1, 1, 1, 1, 1, 1],
    [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
    [120, 200, 120, 250, 260, 150],
])
y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent (naive implementation)
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    with torch.no_grad():
        theta -= learning_rate * theta.grad
```

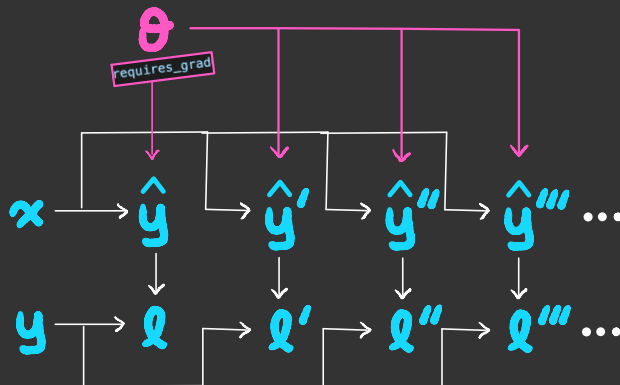


finally we should reset the gradient

```
x = torch.tensor([
    [
        [1, 1, 1, 1, 1, 1],
        [6.4, 6.0, 5.5, 5.0, 6.6, 7.0],
        [120, 200, 120, 250, 260, 150],
    ]
])
y = torch.tensor([140.0, 240.0, 160.0, 320.0, 300.0, 170.0])
theta = torch.tensor([15.0, -10.0, 1.5], requires_grad=True)

# gradient descent
num_steps = 5
learning_rate = 0.000001
for i in range(num_steps):
    y_hat = (theta.unsqueeze(dim=0) @ x).squeeze()
    loss = ((y_hat - y) ** 2).sum()
    loss.backward()

    with torch.no_grad():
        theta -= learning_rate * theta.grad
        theta.grad = None
```



before each call to `.backward()`